TAD - Pilha

Prof. Fernando Garcia

Referências

Material adaptado do livro:

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.

Os código presentes nos slides estão disponíveis no site do autor (link no último slide)

Pilhas

- Existem aplicações para listas lineares nas quais inserções, retiradas e acessos aos itens ocorrem em um dos extremos da lista.

- Pilha é uma lista linear em que todas as inserções, retiradas e, geralmente, todos os acessos são feitos em apenas um extremo da lista.

Pilhas

- Os itens são colocados um sobre o outro. O item inserido mais recentemente está no topo e o inserido menos recentemente no fundo.

- O modelo intuitivo é o de um monte de pratos em uma prateleira:
  - É conveniente retirar ou adicionar pratos na parte superior.

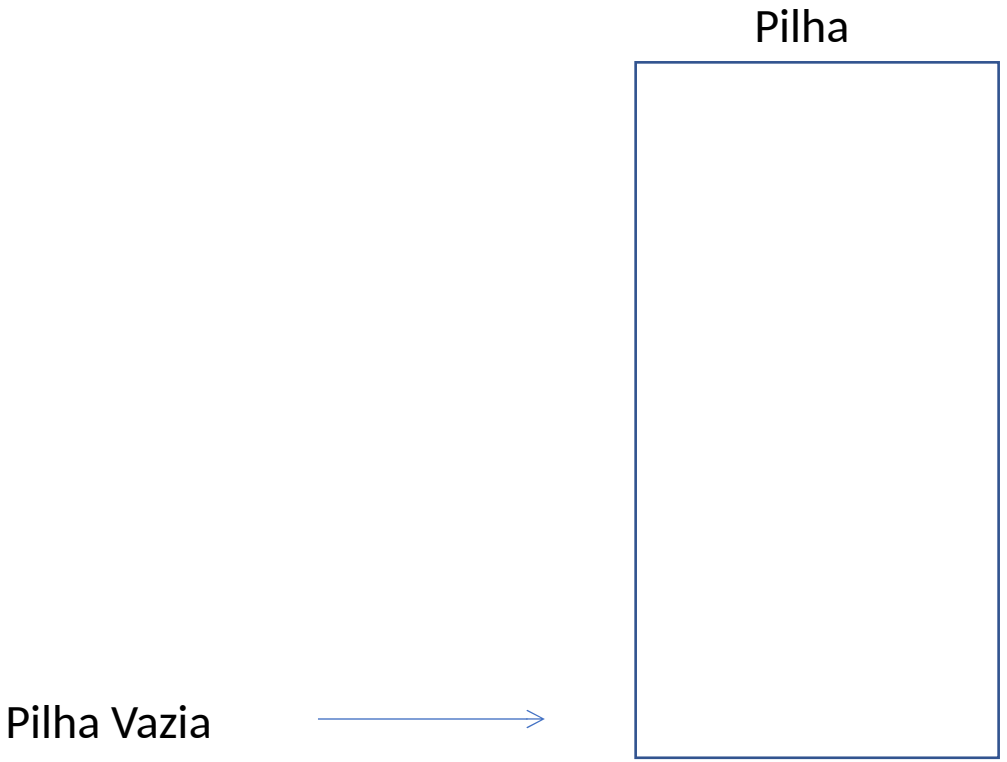
Propriedade das Pilhas

- O último item inserido é o primeiro item que pode ser retirado da lista.
  - Chamadas listas **lifo** (“last-in, first-out”).
- Existe uma ordem linear para pilhas, do “mais recente para o menos recente”.
- É ideal para processamento de estruturas aninhadas de profundidade imprevisível.
  - A ordem de remoção garante que as estruturas mais internas serão processadas antes das mais externas.

Propriedade das Pilhas

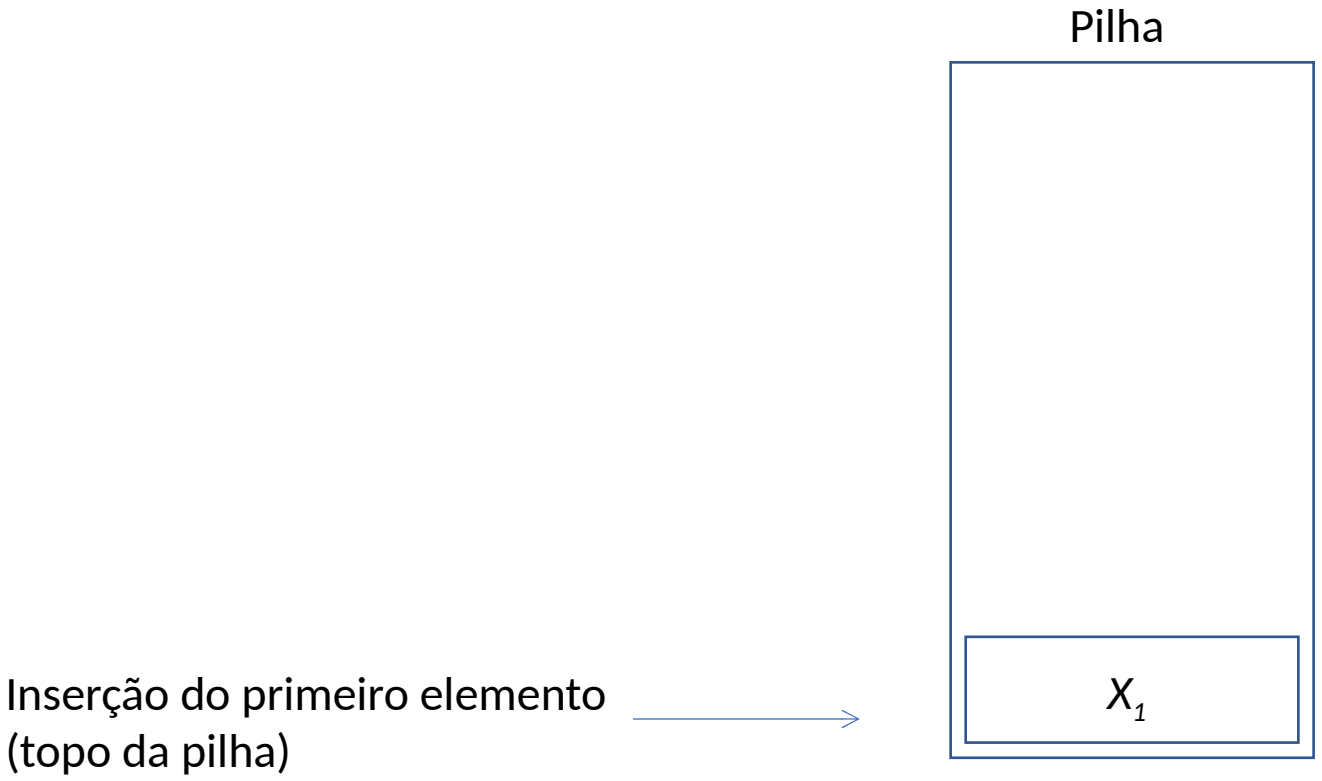
- Aplicações em estruturas aninhadas:
  - Quando é necessário caminhar em um conjunto de dados e guardar uma lista de coisas a fazer posteriormente.
    - O controle de sequências de chamadas de subprogramas.
    - A sintaxe de expressões aritméticas.
- São utilizadas para implementar a recursividade.

Propriedade das Pilhas



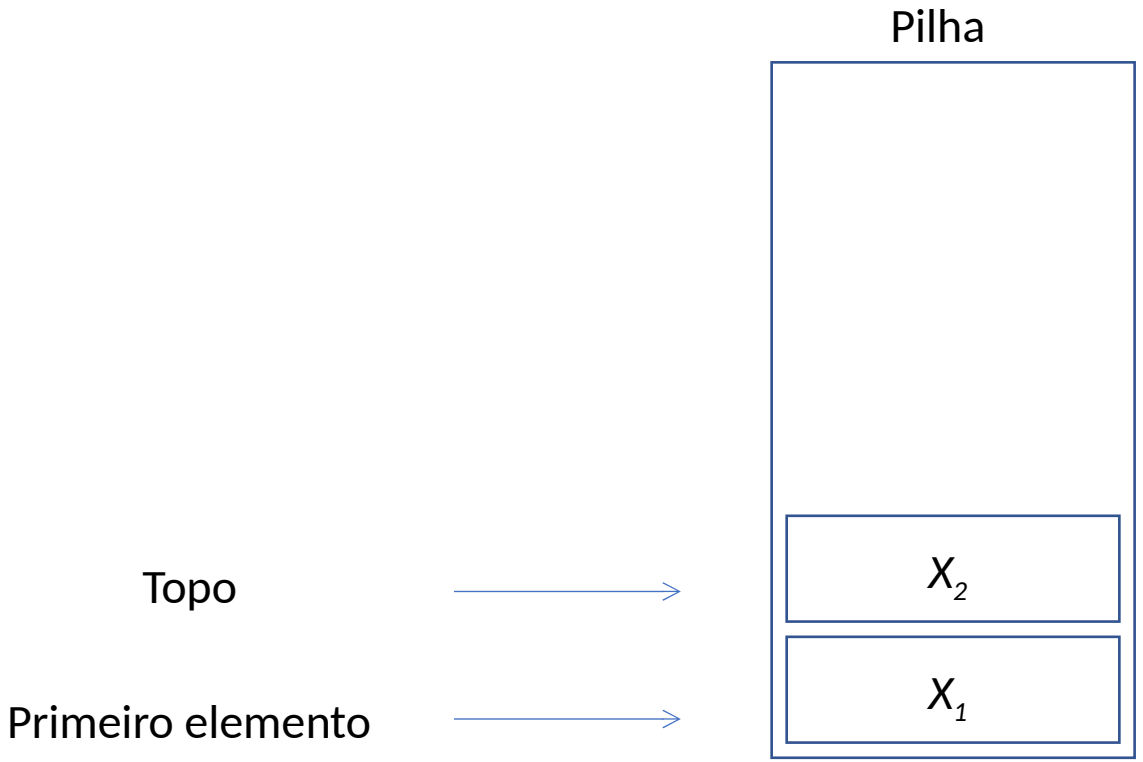
Propriedade das Pilhas

- Inserção de elementos da Pilha



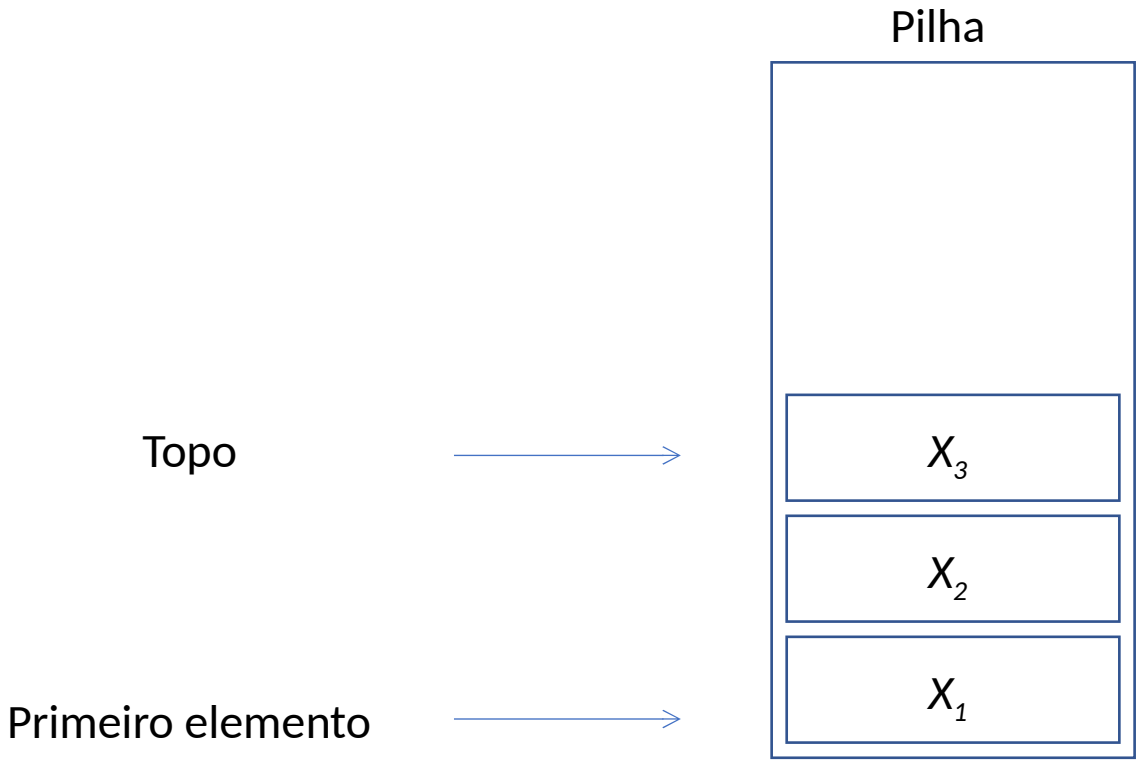
Propriedade das Pilhas

- Inserção de elementos da Pilha



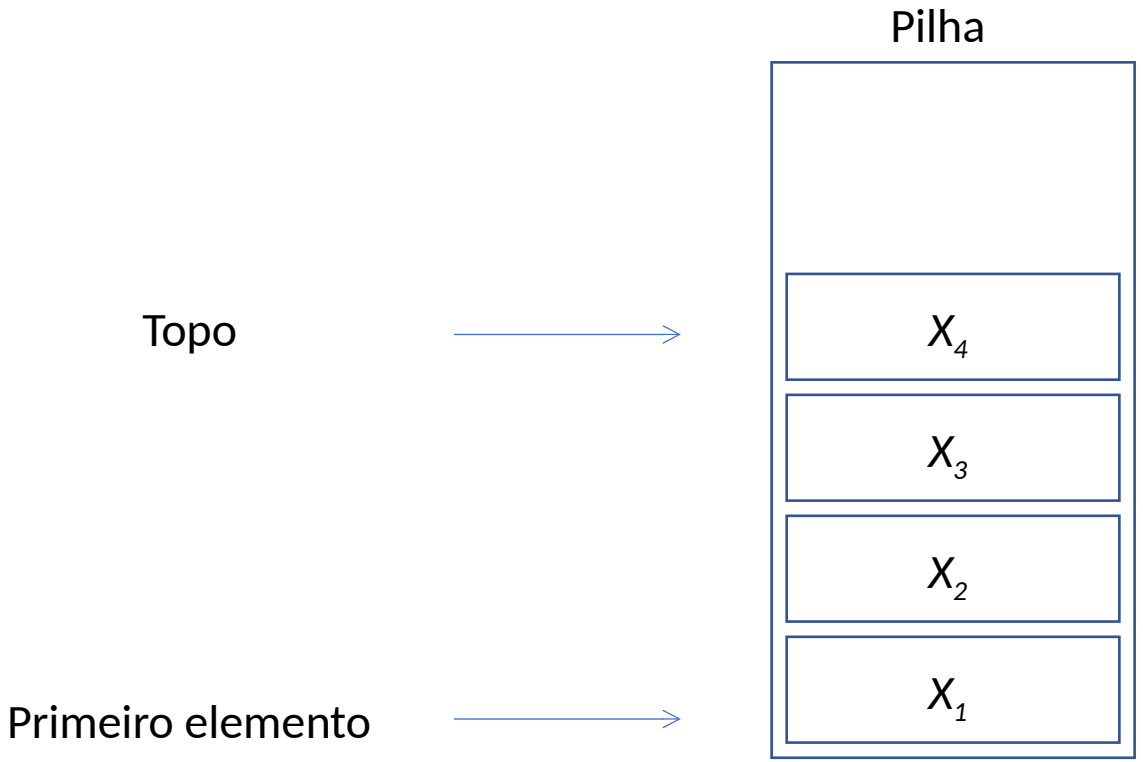
Propriedade das Pilhas

- Inserção de elementos da Pilha



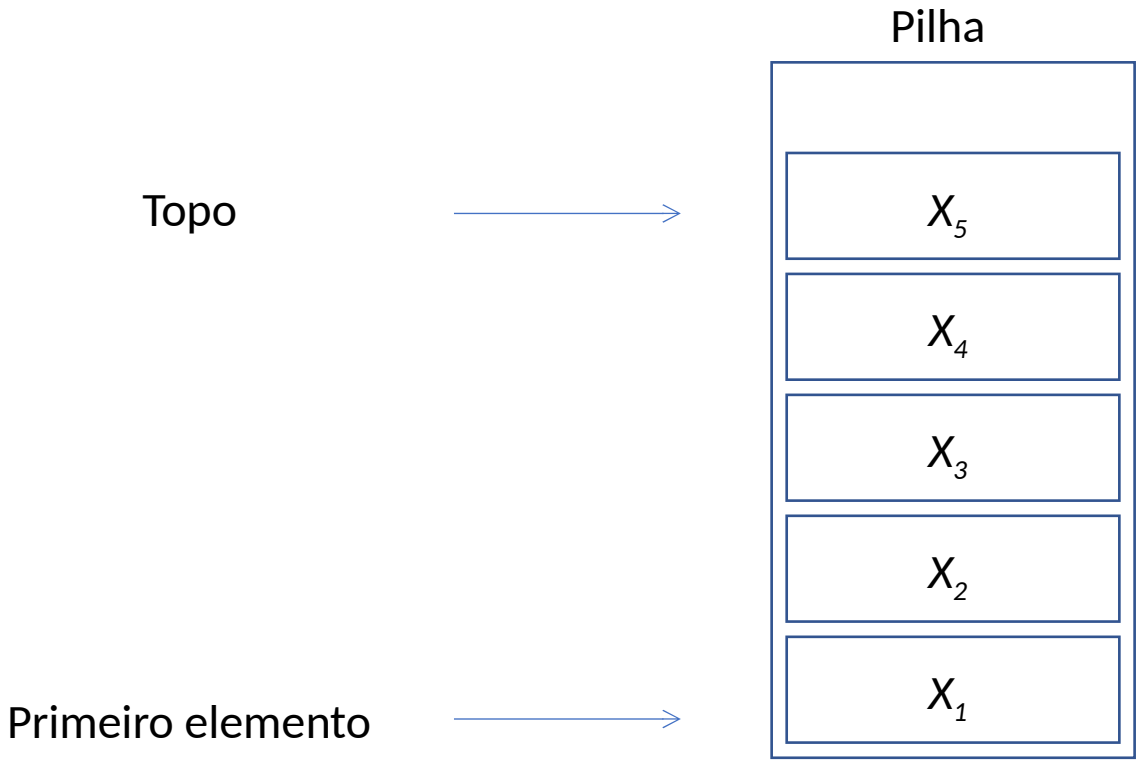
Propriedade das Pilhas

- Inserção de elementos da Pilha



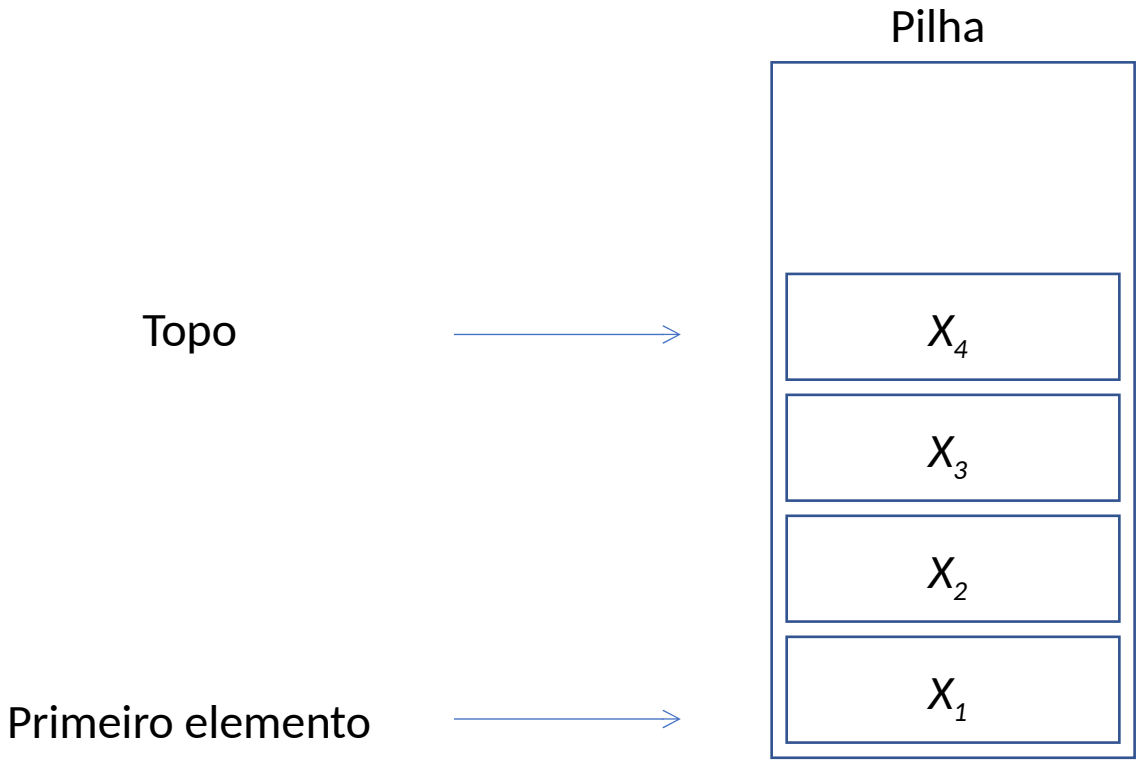
Propriedade das Pilhas

- Inserção de elementos da Pilha



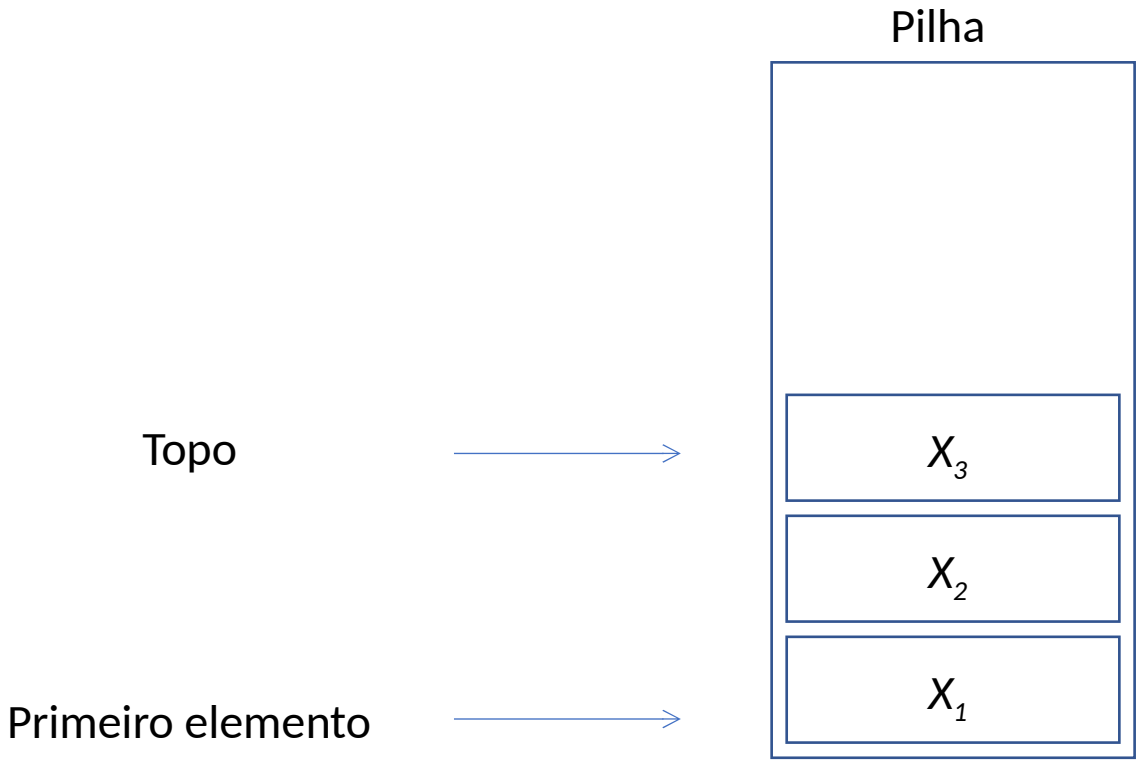
Propriedade das Pilhas

- Remoção de elementos da Pilha



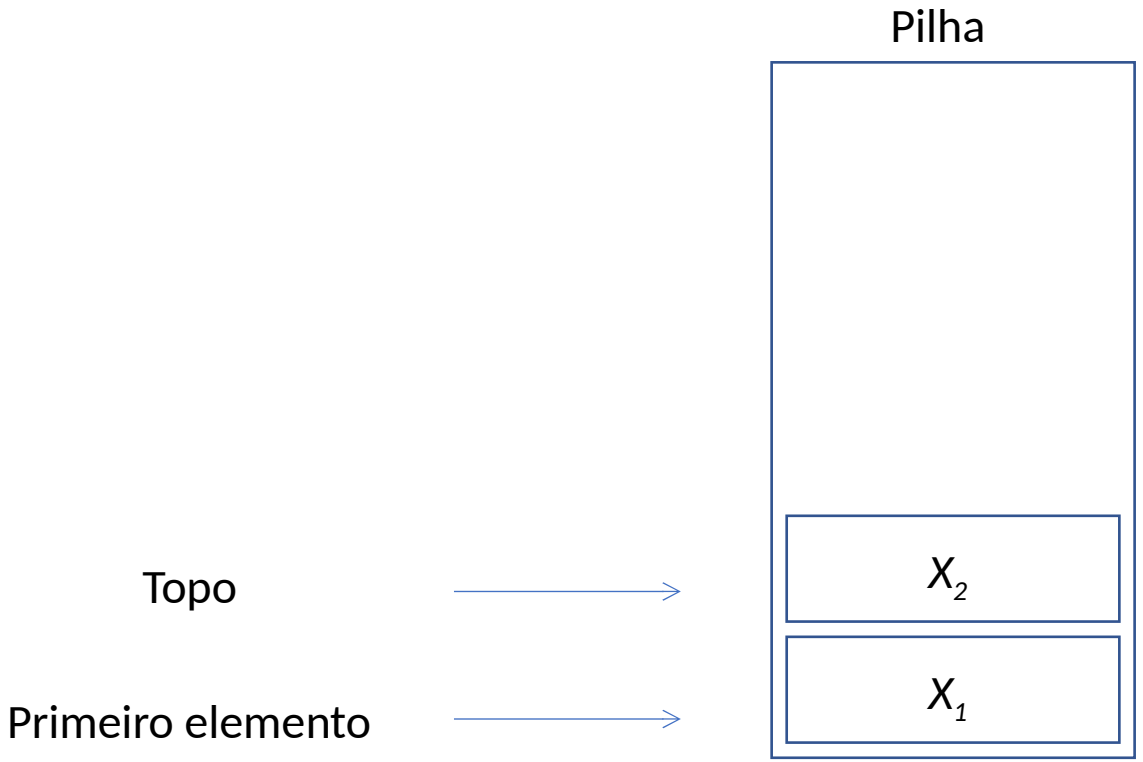
Propriedade das Pilhas

- Remoção de elementos da Pilha



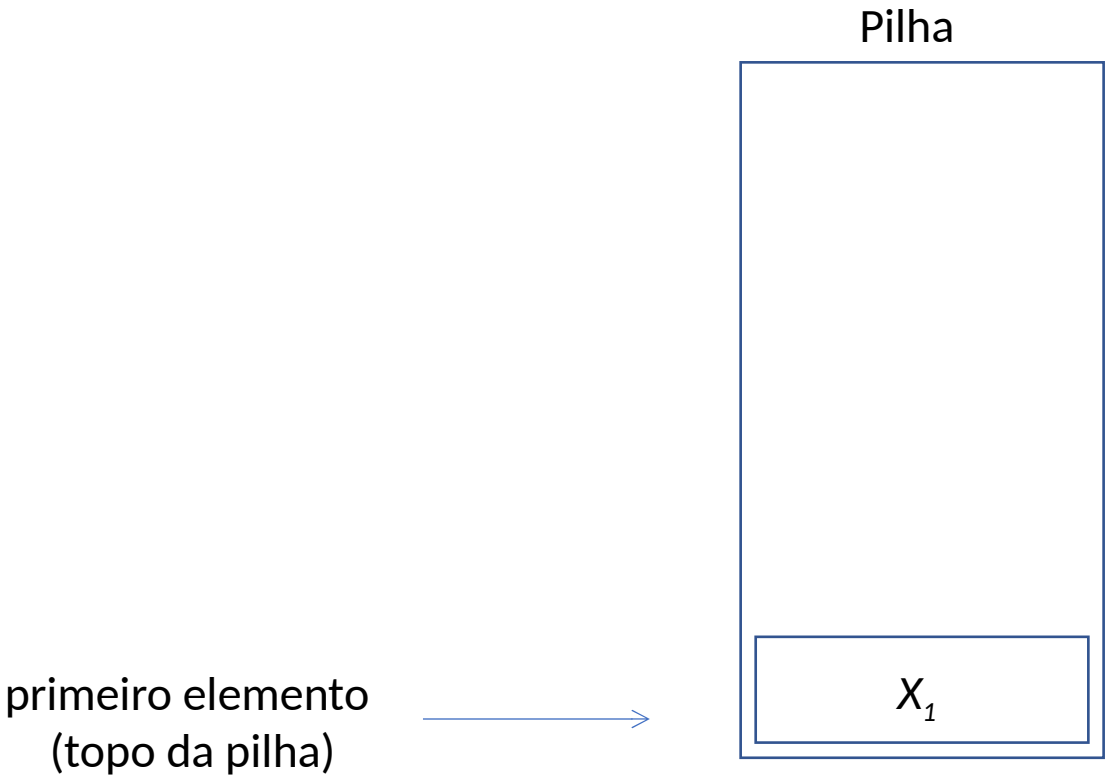
Propriedade das Pilhas

- Remoção de elementos da Pilha



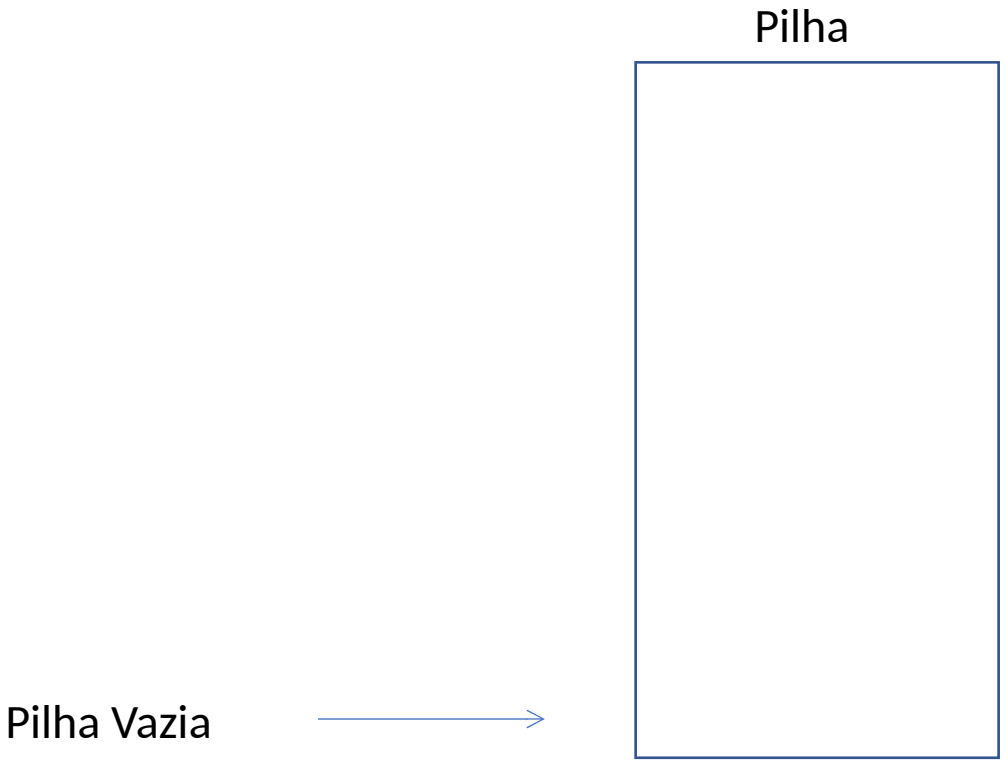
Propriedade das Pilhas

- Remoção de elementos da Pilha



Propriedade das Pilhas

- Remoção de elementos da Pilha



TAD Pilhas

- Conjunto de operações:
  - Cria uma pilha Vazia.
  - Verifica se a lista está vazia. Retorna *true* se a pilha está vazia; caso contrário, retorna *false*.
  - Empilhar o item x no topo da pilha.
  - Desempilhar o item x no topo da pilha, retirando-o da pilha.
  - Verificar o tamanho atual da pilha.

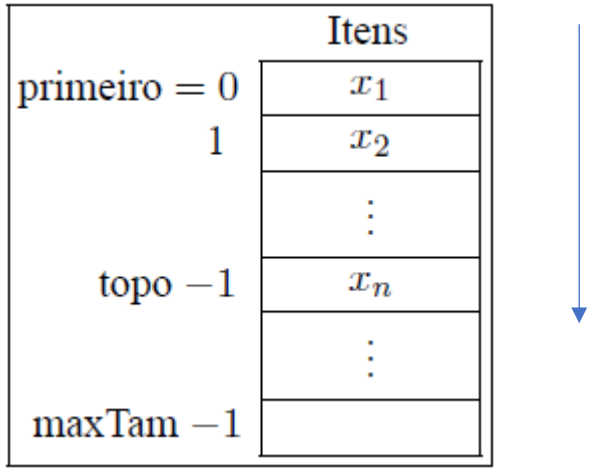
TAD Pilhas

- Existem várias opções de estruturas de dados que podem ser usadas para representar pilhas.
- Representações mais utilizadas:
  - Por meio de *arranjos e*
  - *Estruturas auto referenciadas* (listas ligadas ou encadeadas).

Implementação de Pilhas por meio de Arranjos

- Os itens da pilha são armazenados em posições contíguas de memória
- Inserções e as retiradas ocorrem no topo da pilha:

- Uso de um cursor chamado **Topo** para controlar a posição do item no topo da pilha.



Direção de inserção da pilha da figura ao lado

Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Implementação de Pilhas por meio de Arranjos

- Itens são armazenados em um arranjo de tamanho suficiente para conter a pilha.

- O outro campo do mesmo registro contém uma referência para o item no topo da pilha

- Uma constante define o **tamanho máximo** permitido para a pilha

Implementação de Pilhas por meio de Arranjos

```c
#define CAPACIDADE_MAX 1000

// Definição do tipo de item da pilha
typedef struct {
    int valor;
    // outros campos podem ser adicionados
} Item;

// Estrutura da pilha
typedef struct {
    Item item[CAPACIDADE_MAX];
    int topo;
} Pilha;

// Cria pilha com capacidade fixa
Pilha* criar_pilha() {
    Pilha* p = (Pilha*) malloc(sizeof(Pilha));
    if (!p) {
        fprintf(stderr, "Erro: Falha ao alocar memória para a pilha\n");
        exit(1);
    }
    p->topo = 0;
    return p;
}
```

Implementação de Pilhas por meio de Arranjos

```c
// Empilha um item
void empilha(Pilha* p, Item x) {
    if (p->topo == CAPACIDADE_MAX) {
        fprintf(stderr, "Erro: A pilha está cheia\n");
        exit(1);
    }
    p->item[p->topo++] = x;
}

// Desempilha e retorna o item do topo
Item desempilha(Pilha* p) {
    if (p->topo == 0) {
        fprintf(stderr, "Erro: A pilha está vazia\n");
        exit(1);
    }
    return p->item[--p->topo];
}
```

Implementação de Pilhas por meio de Arranjos

```c
// Verifica se a pilha está vazia
bool pilha_vazia(Pilha* p) {
    return p->topo == 0;
}

// Retorna o número de elementos na pilha
int tamanho_pilha(Pilha* p) {
    return p->topo;
}

// Libera memória da pilha
void destruir_pilha(Pilha* p) {
    if (p) {
        free(p);
    }
}
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

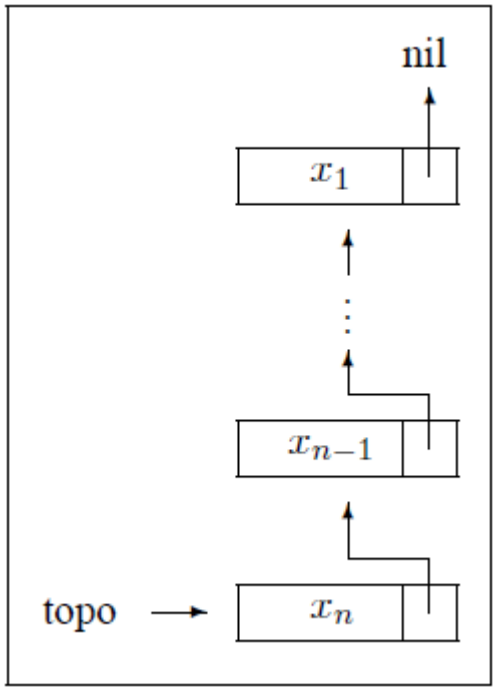
- Ao contrário da implementação de listas lineares, por meio e estruturas auto referenciadas não há necessidade de manter uma célula *head*:
  - Mantem-se o controle pelo topo da pilha.

- Para desempilhar um item, basta “desconectar” a célula que contém x_n e a célula que contém x_{n-1} passa a ser a célula de topo.

- Não há mais controle sobre o tamanho máximo da pilha, que pode crescer enquanto houver memória disponível no sistema.

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

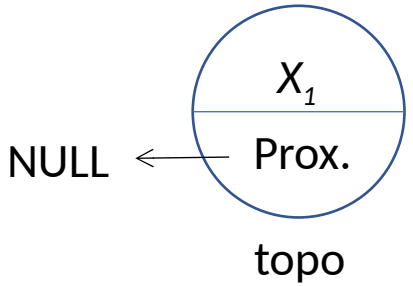
- Para empilhar um novo item, basta fazer a operação contrária, criando uma nova célula para receber o novo item.

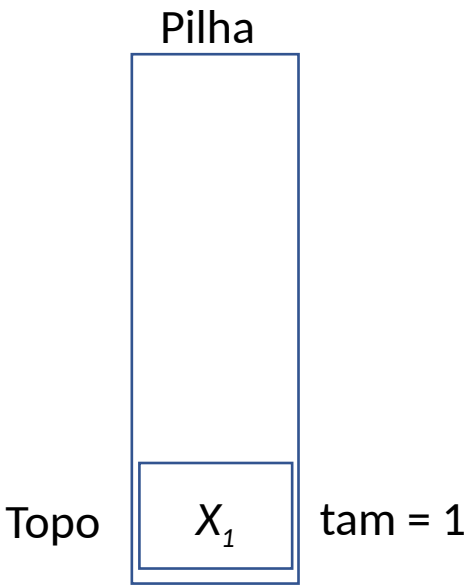


Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

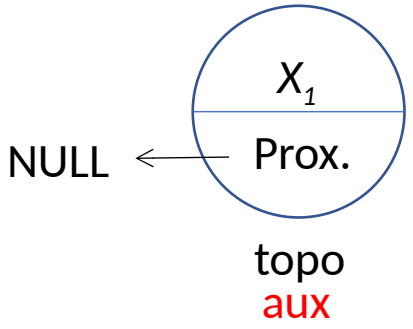
- Pilha com apenas um elemento:

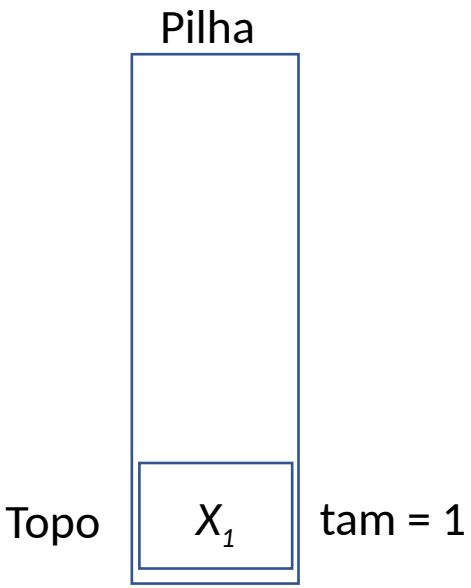




Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

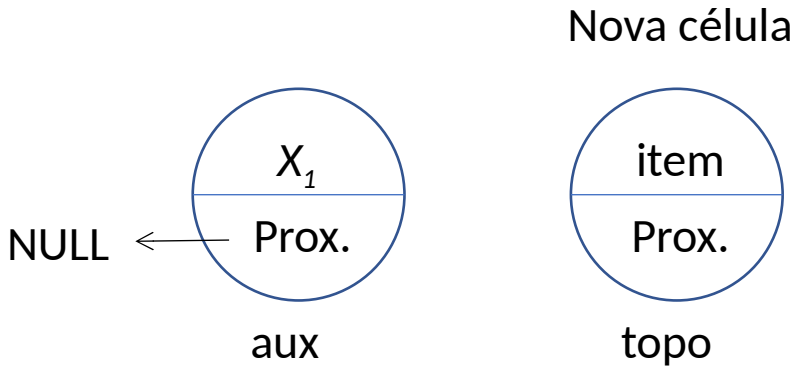


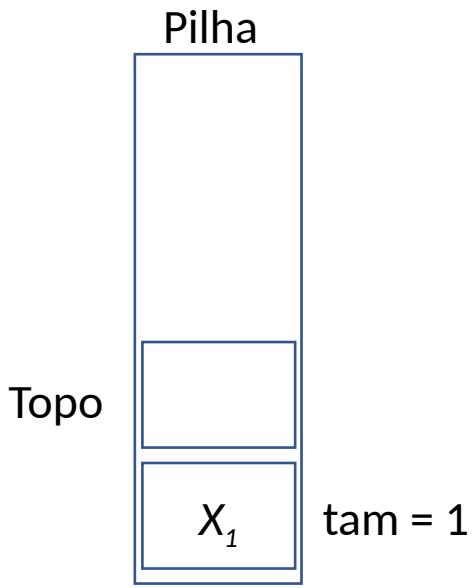



```
void empilha (Item x) {
    Celula *aux = topo;
    topo = (Celula*)
(malloc(sizeof(Celula*)));
    topo->item = x;
    topo->prox = aux;
    tam++;
  }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

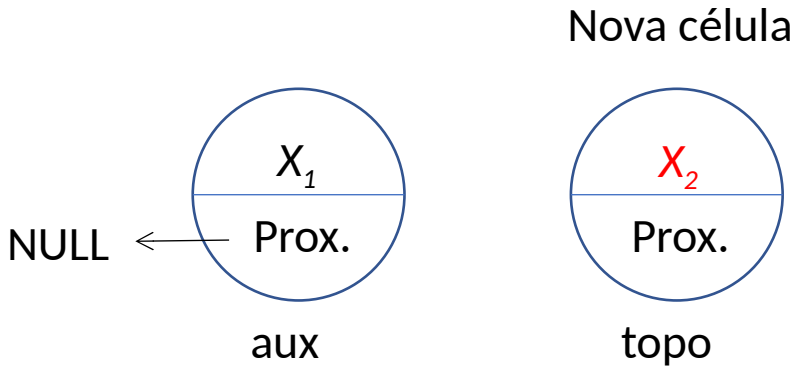


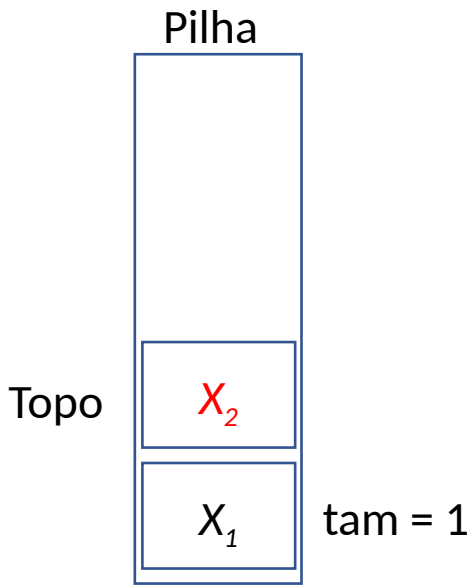



```
void empilha (Item x) {
    Celula *aux = topo;
    topo = (Celula*)
(malloc(sizeof(Celula*)));
    topo->item = x;
    topo->prox = aux;
    tam++;
  }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

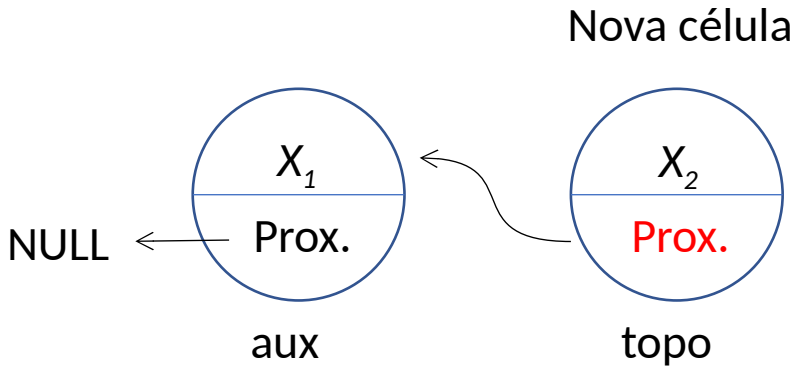


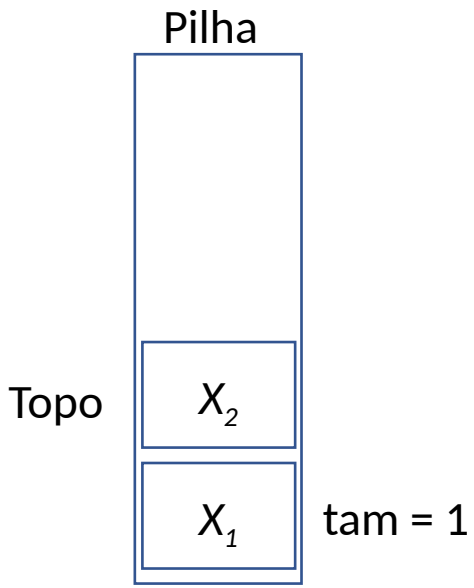



```
void empilha (Item x) {
    Celula *aux = topo;
    topo = (Celula*)
(malloc(sizeof(Celula*)));
    topo->item = x;
    topo->prox = aux;
    tam++;
  }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

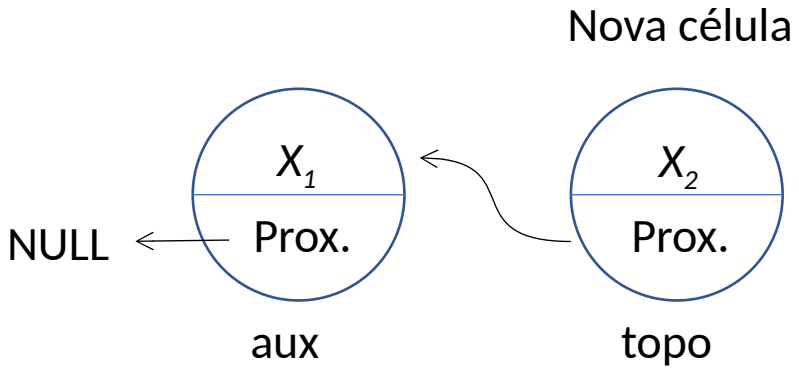


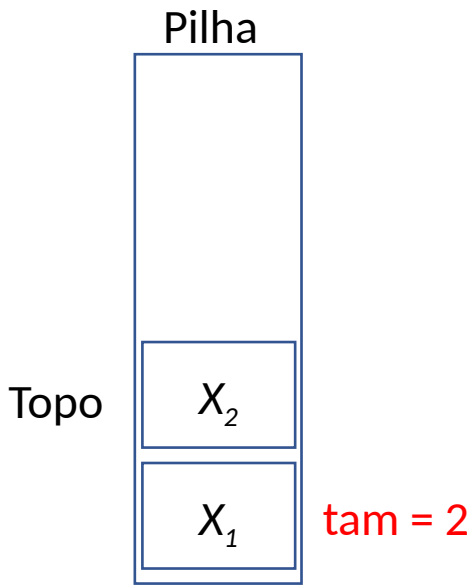



```
void empilha (Item x) {
    Celula *aux = topo;
    topo = (Celula*)
(malloc(sizeof(Celula*)));
    topo->item = x;
    topo->prox = aux;
    tam++;
  }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

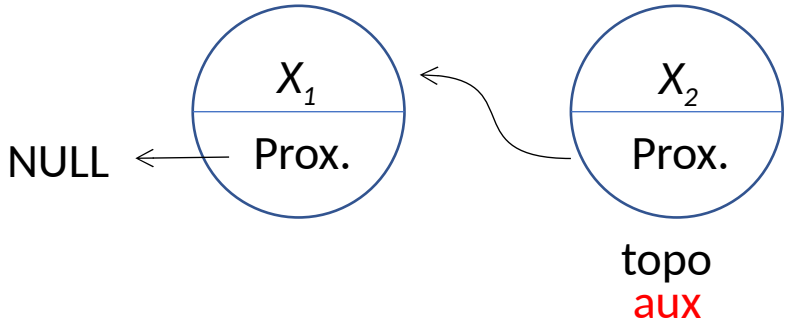


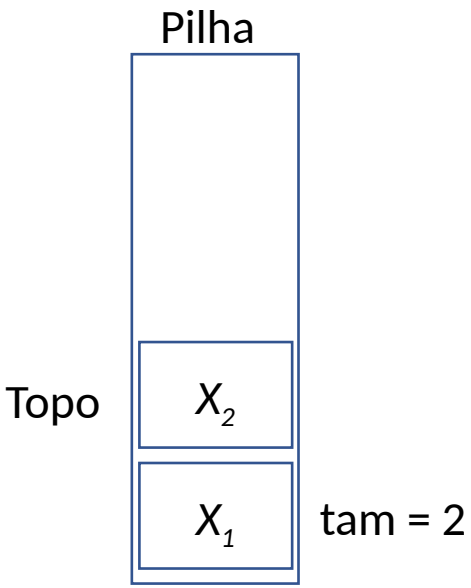



```
void empilha (Item x) {
    Celula *aux = topo;
    topo = (Celula*)
(malloc(sizeof(Celula*)));
    topo->item = x;
    topo->prox = aux;
    tam++;
  }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

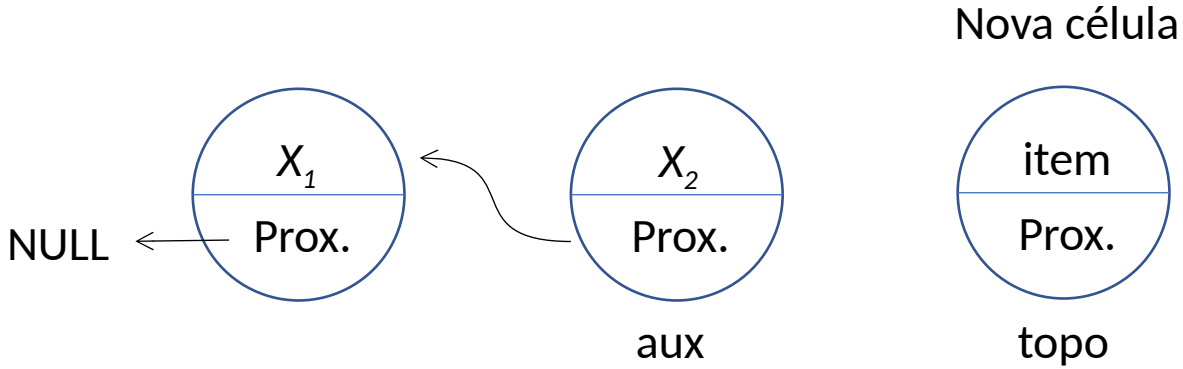
- Inserção de novos elementos na pilha (empilha):

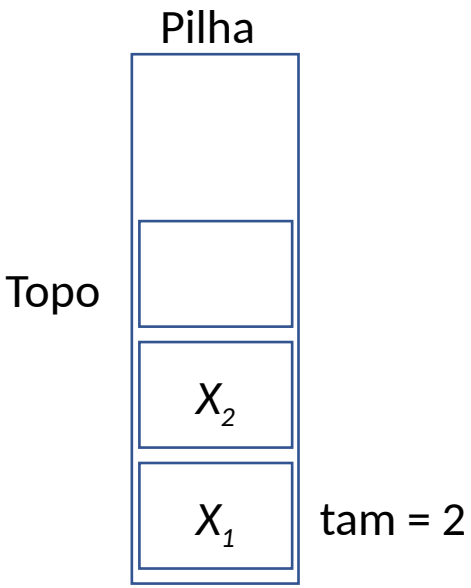




Implementação de Pilhas por meio de Estruturas Auto Referenciadas

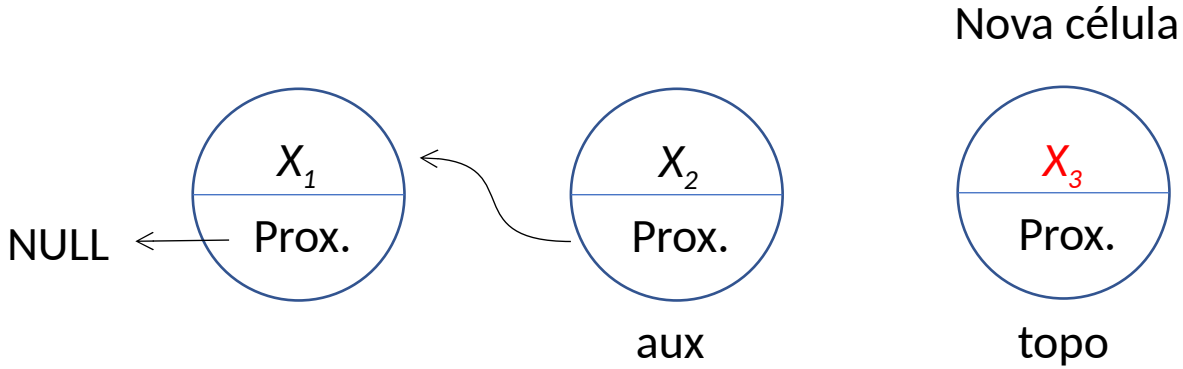
- Inserção de novos elementos na pilha (empilha):

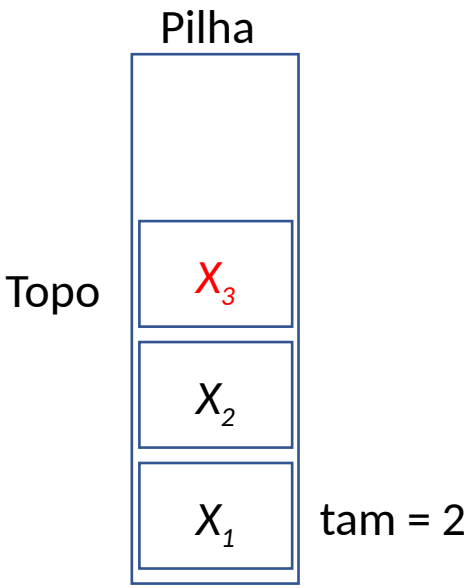




Implementação de Pilhas por meio de Estruturas Auto Referenciadas

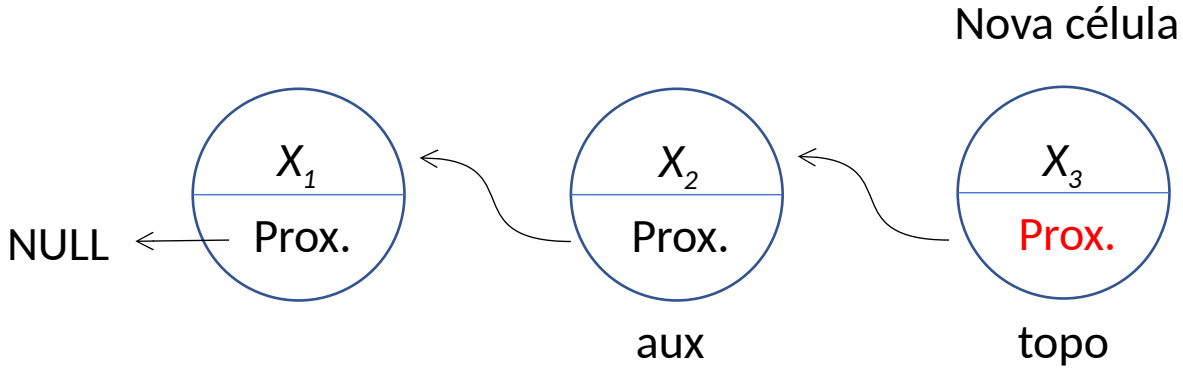
- Inserção de novos elementos na pilha (empilha):

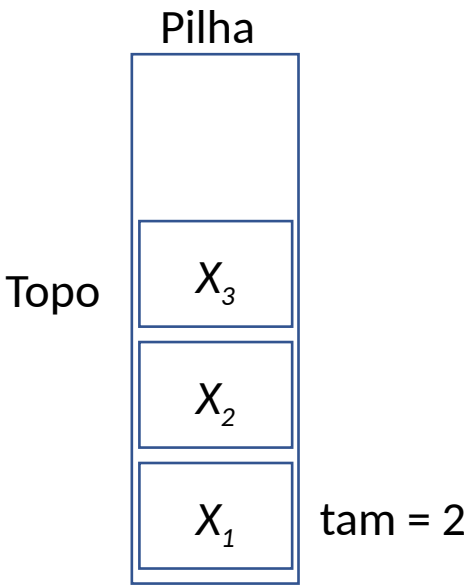




Implementação de Pilhas por meio de Estruturas Auto Referenciadas

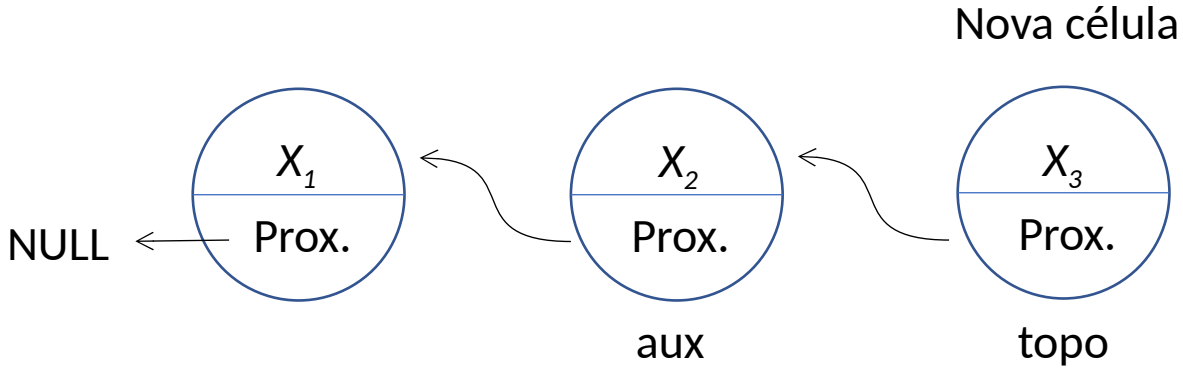
- Inserção de novos elementos na pilha (empilha):

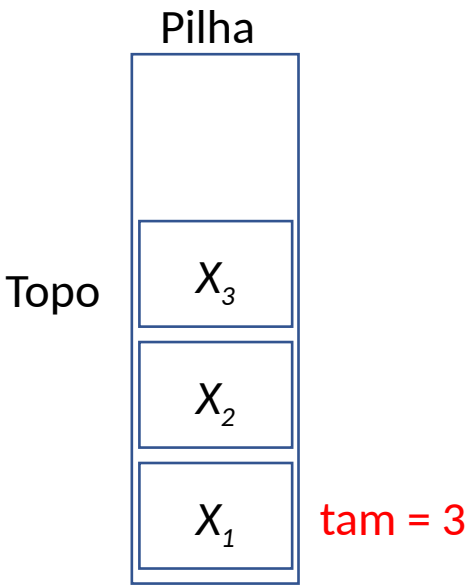




Implementação de Pilhas por meio de Estruturas Auto Referenciadas

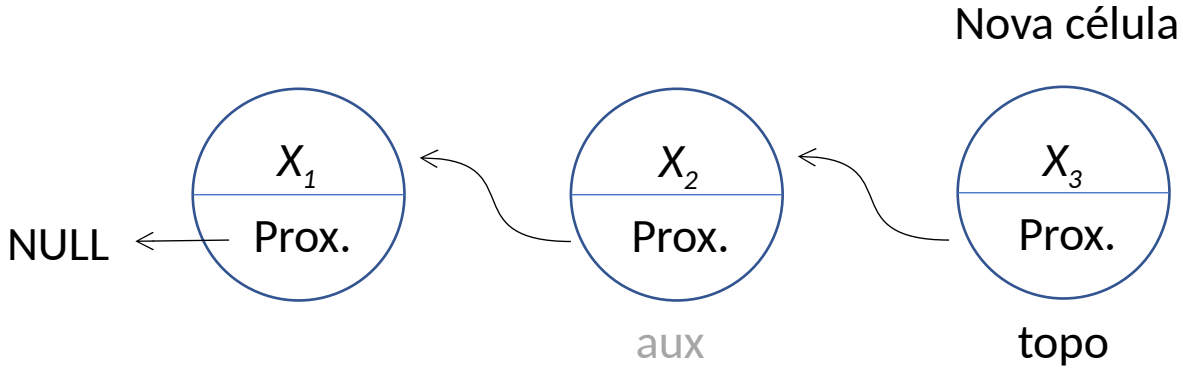
- Inserção de novos elementos na pilha (empilha):

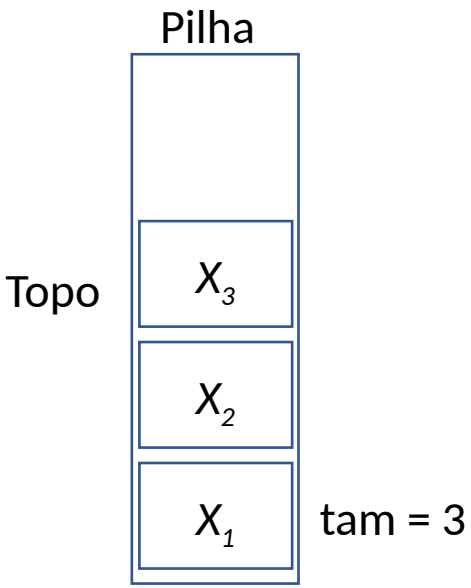




Implementação de Pilhas por meio de Estruturas Auto Referenciadas

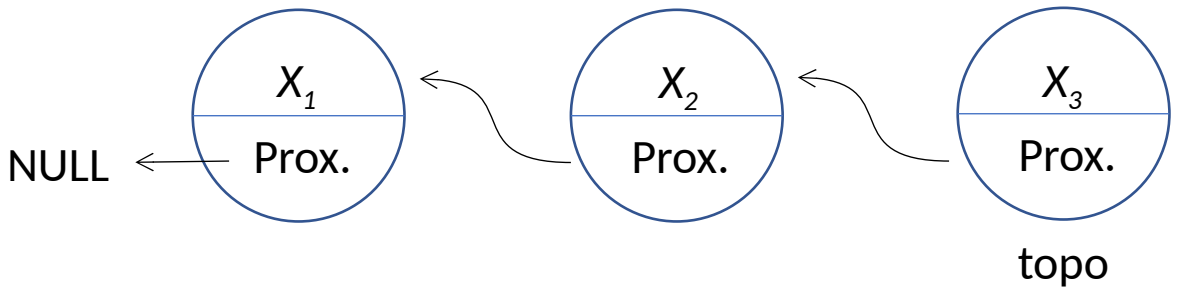
- Inserção de novos elementos na pilha (empilha):

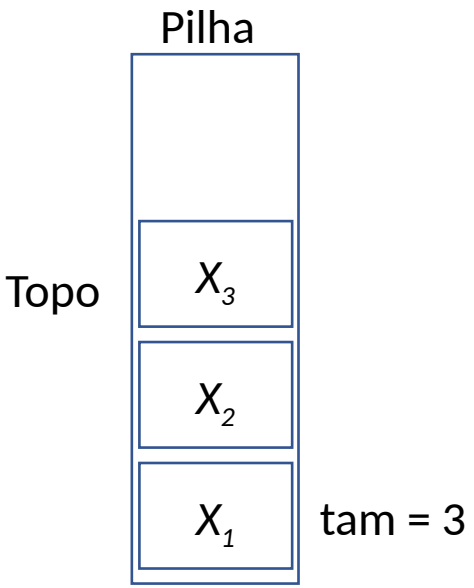




Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Remoção de itens na pilha (desempilha):

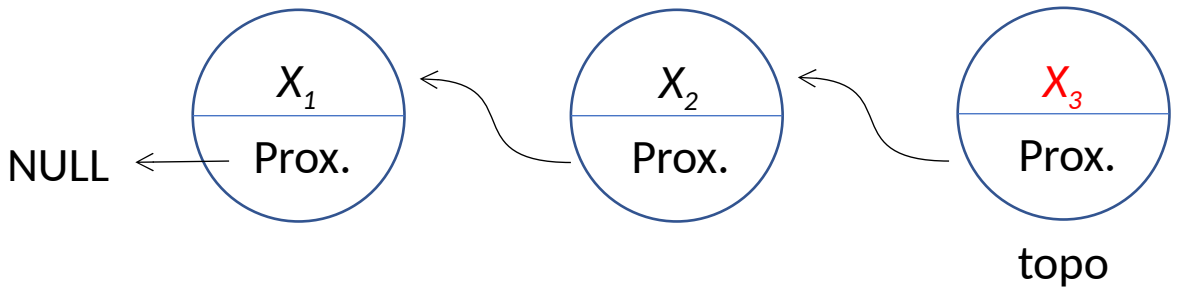


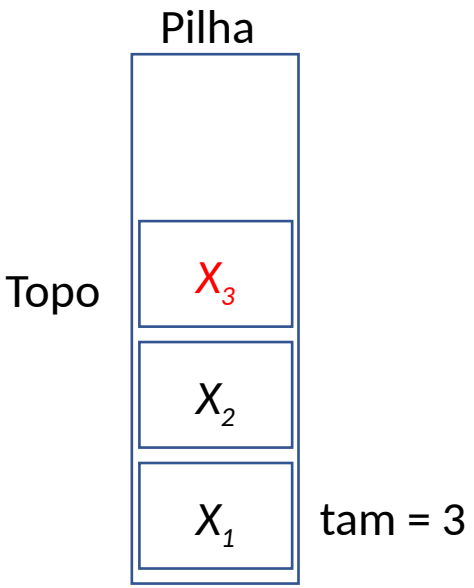



```
Item desempilha (){
    if (vazia ())
      printf("Erro: A pilha esta vazia");
    Item item = topo->item;
    Celula *q = topo; topo = topo->prox; tam--;
    free(q);
    return item;
}
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Remoção de itens na pilha (desempilha):

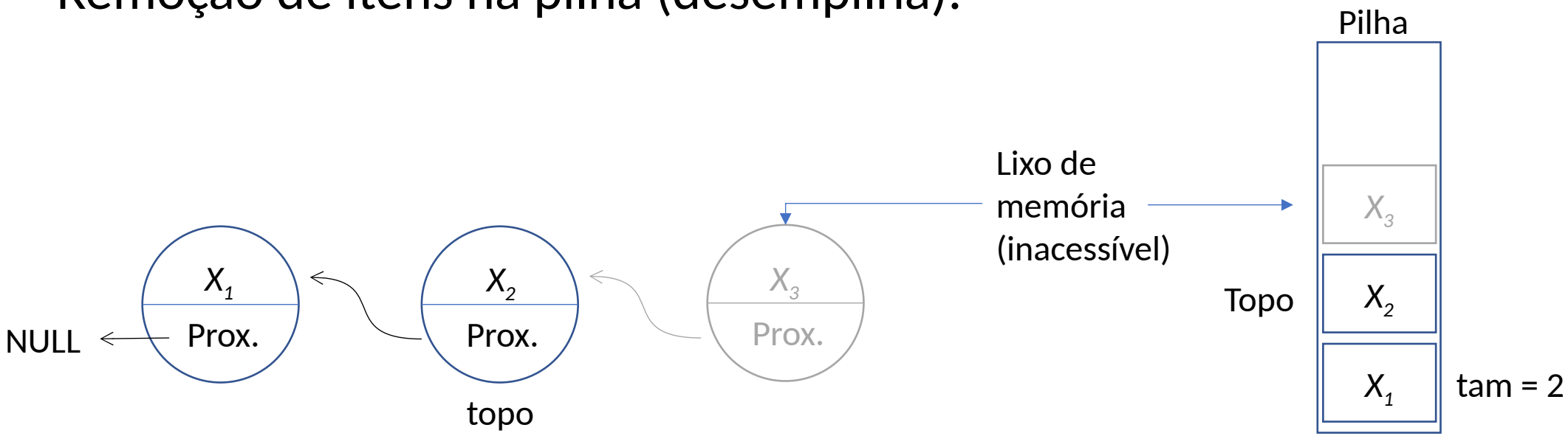





```
Item desempilha (){
    if (vazia ())
      printf("Erro: A pilha esta vazia");
    Item item = topo->item;
    Celula *q = topo; topo = topo->prox; tam--;
    free(q);
    return item;
}
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

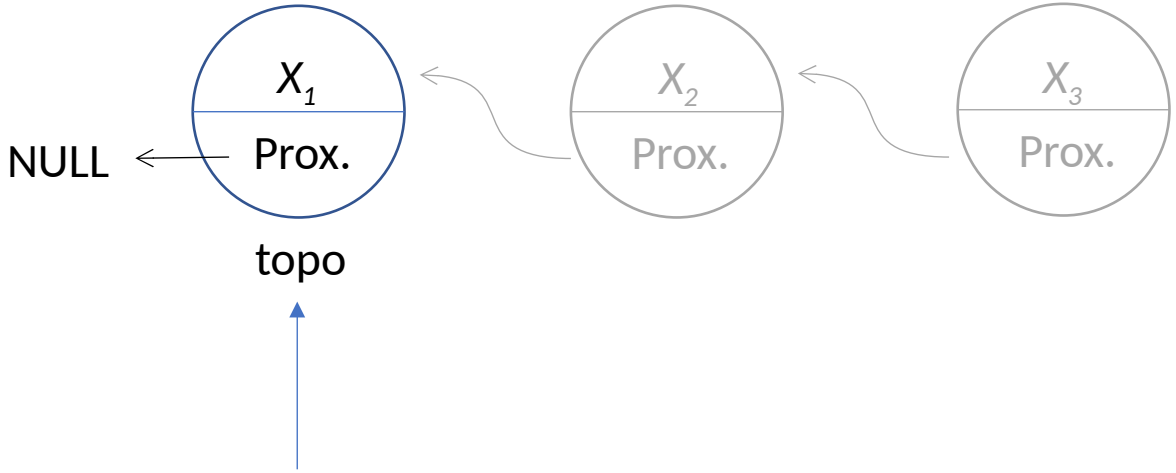
- Remoção de itens na pilha (desempilha):

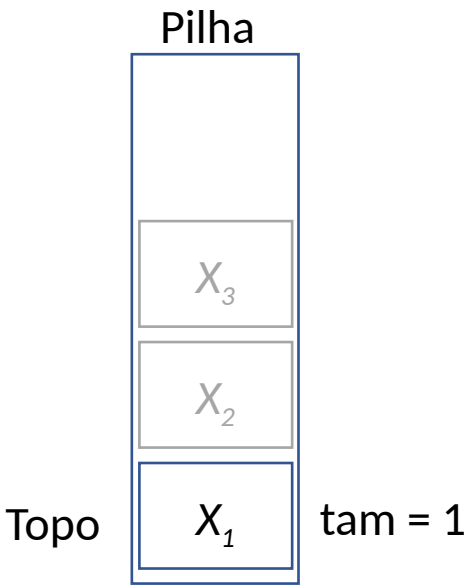



```
Item desempilha (){
    if (vazia ())
      printf("Erro: A pilha esta vazia");
    Item item = topo->item;
    Celula *q = topo; topo = topo->prox; tam--;
    free(q);
    return item;
}
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Remoção de itens na pilha (desempilha):

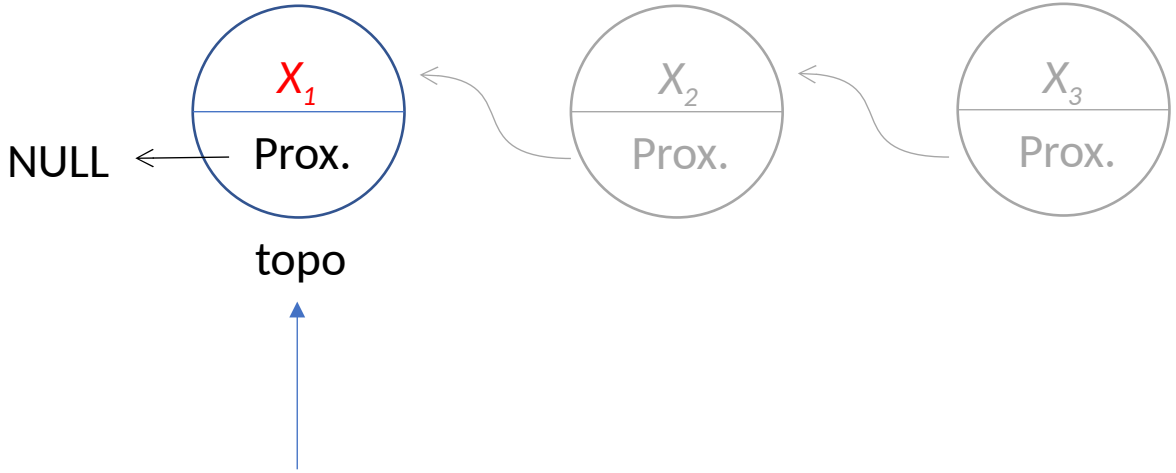


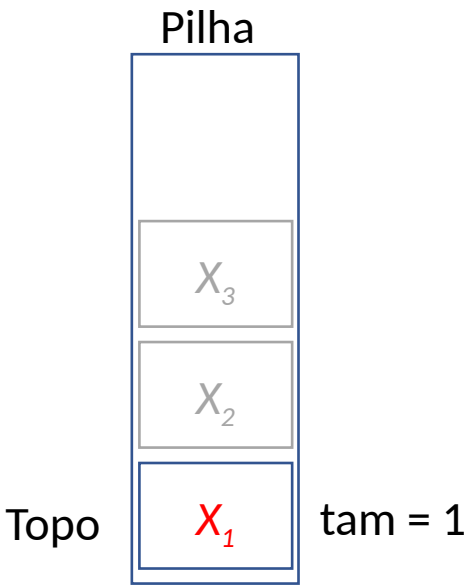


Observe que o primeiro elemento inserido na pilha sempre faz referência para NULL. Quando este elemento for removido, topo passará a referenciar NULL, caracterizando **pilha vazia**.

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Remoção de itens na pilha (desempilha):

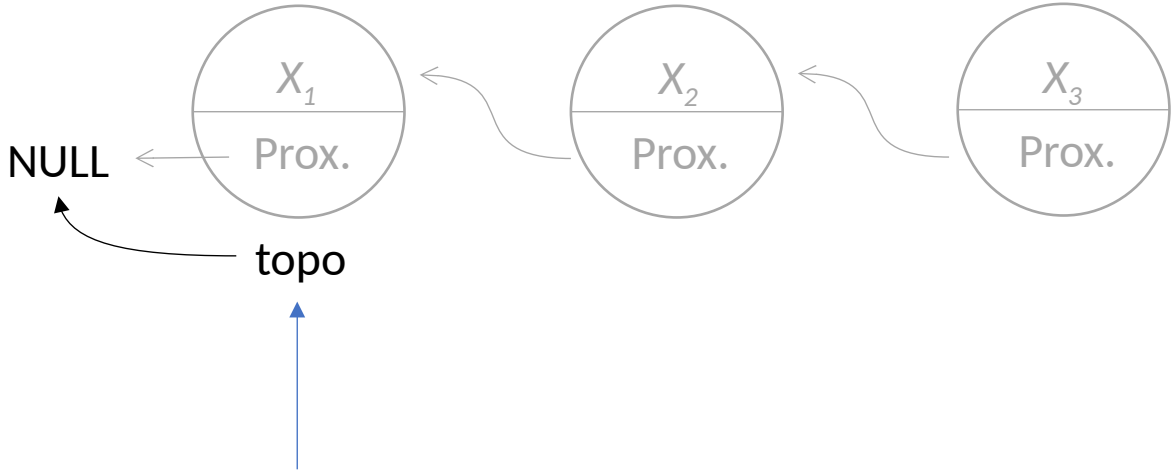


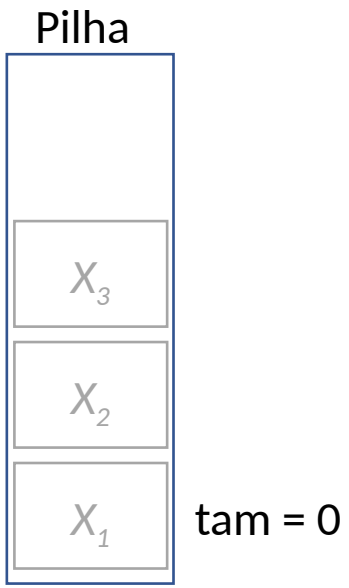


Observe que o primeiro elemento inserido na pilha sempre faz referência para NULL. Quando este elemento for removido, topo passará a referências NULL, caracterizando **pilha vazia**.

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Remoção de itens na pilha (desempilha):





Observe que o primeiro elemento inserido na pilha sempre faz referência para NULL. Quando este elemento for removido, topo passará a referências NULL, caracterizando **pilha vazia**.

Operações sobre Pilhas Usando Estruturas Auto Referenciadas

- O campo *tam* evita a contagem do número de itens no método *tamanho*.
- Cada célula de uma pilha contém um item da pilha e uma referência para outra célula.
- A classe Pilha contém uma referência para o topo da pilha.

Pilhas Usando Estruturas Auto Referenciadas

```c
// Tipo do item armazenado na pilha
typedef struct {
    int valor;
    // outros campos podem ser adicionados aqui
} Item;

// Célula da pilha (lista encadeada)
typedef struct Celula {
    Item item;
    struct Celula* prox;
} Celula;

// Variáveis globais
Celula* topo = NULL; // topo da pilha
int tam = 0;         // número de elementos na pilha
```

Pilhas Usando Estruturas Auto Referenciadas

```c
// Retorna o item do topo sem removê-lo
Item topo_pilha() {
    if (topo == NULL) {
        fprintf(stderr, "Erro: A pilha está vazia\n");
        exit(1);
    }
    return topo->item;
}

// Verifica se a pilha está vazia
bool pilha_vazia() {
    return topo == NULL;
}

// Retorna o número de elementos na pilha
int tamanho_pilha() {
    return tam;
}

// Libera toda a memória usada pela pilha
void destruir_pilha() {
    while (!pilha_vazia()) {
        desempilha();
    }
}
```

Referências

- Baseado no material de aulas de:
  - ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.
  - Para downloads dos códigos em Java:
  - http://www2.dcc.ufmg.br/livros/algoritmos-java/implementacoes-03.php

- Outras referências:
  - CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002. 916p. ISBN 8535209263 (Disponível no Acervo).

  - DROZDEK, Adam. Estrutura de dados e algoritmos em C++. São Paulo: Pioneira Thomson Learning, 2002. xviii, 579p. ISBN 8522102953 (Disponível no Acervo).